

Vietnam National University, Ho Chi Minh City
UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



REPORT SEMINAR

Database Testing for EShop

Instructor:

Hồ Tuấn Thanh

Students:

23127081 - Nguyễn Phan Hùng Linh

23127172 - Nguyễn Chí Đức

23127487 - Thôi Đặng Trường Thọ

Ho Chi Minh City, July 2026

Table of Contents

Database Testing for EShop

Introduction	3
Tool Overview	3
Installation and Setup	4
Hands-on Practice and Demo Steps	4
EShop Scenario Tested	6
Comparison	7
Strengths and Limitations	8
Problems Encountered	9
Lessons Learned	9
Conclusion	9
Evidence and Appendix	11
References	18

1. Introduction

Database testing is important because many application bugs are not only in the user interface. They happen when data is saved, updated, deleted, or checked in the database.

For this seminar, our System Under Test (SUT) is the EShop project in <https://github.com/ttbhanh/eshop-sut>

EShop is a good SUT because it is a normal shopping application. It has users, products, categories, coupons, carts, orders, and admin features. These features all depend on correct database behavior.

The three tools studied in this report are:

- DbUnit
- Database Rider
- Tonic.ai

The main goal is to understand how these tools help us prepare test data, check database results, and create safer test data.

2. Tool Overview

Tool	Main purpose	Used for
DbUnit	Setup database by dataset file and check data	Database unit and integration test
Database Rider	Make DbUnit-style tests easier to write	Easier database test maintenance
Tonic.ai	Create safe test data and mask sensitive data	Test data management

DbUnit

DbUnit is a Java testing library. It can load a dataset into a database before a test runs. After the test runs, we can check if the database contains the expected data.

In our demo, DbUnit used an XML dataset.

Database Rider

Database Rider is built on top of DbUnit. It gives a cleaner way to write database tests. It can use YAML datasets, which are easier to read than XML.

In our demo, Database Rider used a YAML dataset.

Tonic.ai

Tonic.ai is different from DbUnit and Database Rider. It is not mainly an assertion tool. It is used to create generated or masked data for testing.

For EShop, this is useful because the users table can contain sensitive data such as name, email, password, phone, and address.

3. Installation and Setup

The basic setup used for the seminar was:

1. Install Node.js for the EShop backend and frontend.
2. Install Java and Maven for DbUnit and Database Rider demos.
3. Install SQLite tools for checking the EShop database.
4. Clone or open the EShop project.
5. Run `npm install` in the Node.js parts of the project.
6. Run `node database.js` to create and seed the SQLite database.
7. Create a DbUnit demo Maven project.
8. Create a Database Rider demo Maven project.
9. Prepare a Tonic.ai workspace or Tonic.ai generated-data workflow.

The EShop backend database is:

```
eshop-sut/backend/database.sqlite
```

The EShop backend uses SQLite. The main database seed file is:

```
eshop-sut/backend/database.js
```

The database includes tables such as:

- users
- categories
- products
- orders
- order_items
- coupons

4. Hands-on Practice and Demo Steps

4.1 DbUnit Demo

For DbUnit, we created a small Maven test project.

Main files:

```
dbunit-demo/pom.xml
dbunit-demo/src/test/resources/datasets/initial-dataset.xml
dbunit-demo/src/test/java/com/eshop/dbunit/EshopDbUnitTest.java
```

The XML dataset contained sample data for tables such as users, categories, and products.

The test used DbUnit to:

1. Read the XML dataset.
2. Reset the database with `CLEAN_INSERT`.
3. Query the database with JDBC.
4. Assert that the expected data existed.

Command:

```
mvn test
```

Observed result:

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0  
BUILD SUCCESS
```

This means the DbUnit demo passed.

4.2 Database Rider Demo

For Database Rider, we created another small Maven test project.

Main files:

```
database-rider-demo/pom.xml  
database-rider-demo/src/test/resources/datasets/eshop-users.yml  
database-rider-demo/src/test/java/com/eshop/databaserider/EshopDatabaseRiderTest.java
```

The YAML dataset contained sample users, categories, and products.

The test used Database Rider to:

1. Read the YAML dataset.
2. Load the dataset into SQLite.
3. Query the users table.
4. Assert that the table had the expected number of rows.

Command:

```
mvn test
```

Observed result:

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0  
BUILD SUCCESS
```

This means the Database Rider demo passed.

4.3 Tonic.ai Demo

The planned Tonic.ai Structural workflow was:

1. Reset the EShop SQLite database.
2. Export tables such as users, products, coupons, and orders to CSV.
3. Upload CSV files to Tonic.ai.
4. Create file groups.
5. Configure generators for sensitive columns.
6. Run data generation.
7. Download generated CSV files.
8. Compare the original data with the generated data.

Important sensitive columns in the users table:

Column	Why it is sensitive
name	Real user name
email	Personal identifier
password	Login secret
shipping_address	Personal address
phone	Personal phone number

Columns such as id, role, and foreign keys should usually be kept stable, because the application needs them to preserve relationships between tables.

In the seminar demo, Tonic.ai was used for the users data. The main check was to compare the original CSV with the generated CSV and confirm that sensitive values were changed.

5. EShop Scenario Tested

The seminar demo focused on simple database testing scenarios that match the three tools.

Scenario A: Seed and Check Users

For DbUnit and Database Rider, the scenario was:

Load known test data into the EShop SQLite database and check the users table.

This scenario is useful because login, checkout, and profile features all need valid users. If the users table is wrong, many EShop features cannot work.

Expected result:

1. The database is reset to a known state.
2. The dataset creates two users: one admin user and one normal test user.
3. The test query confirms that the users table has the expected data.

DbUnit did this with an XML dataset:

```
src/test/resources/datasets/initial-dataset.xml
```

Database Rider did the same idea with a YAML dataset:

```
src/test/resources/datasets/eshop-users.yml
```

Scenario B: Checkout and Order Check

The EShop application demo scenario was:

```
Login as a test user, add a product to cart, checkout, then check the orders table.
```

Expected result after checkout:

1. The orders table has a new order.
2. The user_id belongs to the logged-in test user.
3. The total_amount matches the cart total.
4. The default order status is pending.

The database can be checked with SQL such as:

```
SELECT COUNT(*) FROM orders;  
SELECT user_id, total_amount, status FROM orders;
```

Scenario C: Mask Sensitive User Data

For Tonic.ai, the scenario was:

```
Export users from SQLite to CSV, upload the CSV to Tonic.ai, mask sensitive columns, and  
download generated CSV.
```

Expected result:

1. name, email, password, shipping_address, and phone are changed.
2. The generated values still look like realistic test data.
3. Important relationship columns such as id are kept stable.
4. The generated data can be used as safer test data for a demo or staging environment.

6. Comparison

Criteria	DbUnit	Database Rider	Tonic.ai
Tool type	Testing library	Testing library wrapper	Test data platform
Common input	XML dataset	YAML dataset	CSV or database source
Main output	Test result	Test result	Generated data
Ease of use	Medium	Easier than DbUnit	Easy after learning the UI
Can assert database results	Yes	Yes	No, not by itself
Can mask sensitive data	No	No	Yes
Best use in EShop	Reset and check database state	Cleaner database tests	Safer generated test data

DbUnit and Database Rider are used to check if the database is correct. Tonic.ai is used to create safer test data. Because of this, the tools support each other, but they do not replace each other.

7. Strengths and Limitations

DbUnit

Strengths:

- It gives strong control over database state.
- It is good for learning the basic idea of database testing.
- It can reset data before tests.

Limitations:

- XML datasets can be long.
- The setup code is more low-level.
- Students must understand Java, JDBC, and database tables.

Database Rider

Strengths:

- YAML datasets are easier to read.
- It reduces some setup code compared with plain DbUnit.
- It is good for maintainable database tests.

Limitations:

- It still requires Java and Maven.
- It still requires knowledge of DbUnit concepts.
- It is not a replacement for understanding the database schema.

Tonic.ai

Strengths:

- It can mask sensitive data.
- It can generate test data that looks realistic.
- It is useful when real production data should not be used directly.

Limitations:

- It is not a test assertion tool.
- It needs correct generator configuration.
- If IDs or foreign keys are changed incorrectly, relationships can break.
- If emails or passwords are masked, default demo login accounts may stop

working.

8. Problems Encountered

During the work, these problems were recorded:

Problem	Simple explanation
Missing <code>sqlite3</code> module	The Node.js dependency was not installed yet
Maven <code>pom.xml</code> parse error	Extra invalid text was accidentally copied into <code>pom.xml</code>
DbUnit order test failed	The expected order did not exist yet
Tonic.ai multiple schemas issue	Different CSV schemas were uploaded into one file group
<code>orders.csv</code> was empty	No orders existed after reset seed data
Masked email/password affected login	Generated credentials may not match the demo login flow
Database Rider import/package issue	Some Java import names had to match the Database Rider version

9. Lessons Learned

Database testing needs stable data. If the database changes randomly, tests become hard to trust.

DbUnit and Database Rider are useful when we want to prepare a known database state before testing. They are also useful when we want to check the database after an action.

Tonic.ai is useful when we need test data but should not use real sensitive data. It helps protect privacy by replacing values such as names, emails, addresses, and phone numbers.

Foreign key relationships are important. For example, if `orders.user_id` points to `users.id`, we should be careful when generating or changing IDs.

Database tests can check things that UI screenshots alone cannot prove. For example, after checkout, the UI may show success, but the database test can confirm that a row was really added to the `orders` table.

10. Conclusion

DbUnit is useful for understanding the foundation of database testing. It shows how to load a dataset and check database results.

Database Rider is easier to maintain because YAML datasets are cleaner and the testing workflow is shorter.

Tonic.ai is useful for safe test data. It helps generate or mask data so testing does not need to expose sensitive information.

For EShop, the three tools work best together:

- DbUnit and Database Rider help verify database behavior.
- Tonic.ai helps prepare safer test data.
- The EShop checkout scenario shows why checking the database after a user action is useful.

11. Evidence and Appendix

Supporting documents:

- User Guide
- DbUnit Step-by-Step Guide
- Database Rider Step-by-Step Guide
- Tonic.ai Testing Guide
- AI Audit Report

Seminar screenshot evidence:

DbUnit Evidence

DbUnit evidence 1

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.371 s -- in com.eshop.dbunit.EshopDbUnitTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 0.953 s
[INFO] Finished at: 2026-07-03T22:44:51+07:00
[INFO] -----
jduckiess@192 dbunit-demo %
```

DbUnit evidence 2

EShop

Giỏ hàngChào, Test UserThoát

Thanh toán thành công!

Cảm ơn bạn đã mua sắm tại EShop.

[Quay lại trang chủ](#)

© 2026 EShop SUT. Dành cho mục đích kiểm thử.

DbUnit evidence 3

```
dbunit-demo > src > test > java > com > eshop > dbunit > J EshopDbUnitTest.java
18  class EshopDbUnitTest {
24      void shouldLoadInitialDatasetAndCheckUsers() throws Exception {
37
38          try (Connection connection = DriverManager.getConnection("jdbc:sqlite:" + DB_PATH);
39              Statement statement = connection.createStatement();
40              ResultSet resultSet = statement.executeQuery("SELECT COUNT(*) AS total FROM users")) {
41
42              assertEquals(2, resultSet.getInt("total"));
43          }
44      }
45      @Test
46      void shouldCheckOrdersAfterCheckout() throws Exception {
47          try (Connection connection = DriverManager.getConnection("jdbc:sqlite:" + DB_PATH);
48              Statement statement = connection.createStatement();
49              ResultSet resultSet = statement.executeQuery("SELECT COUNT(*) AS total FROM orders")) {
50
51              int totalOrders = resultSet.getInt("total");
52
53              System.out.println("Total orders: " + totalOrders);
54
55              // Sau khi checkout thành công, totalOrders nên lớn hơn 0
56              assertEquals(1, totalOrders);
57          }
58      }
59  }
```

DbUnit evidence 4

```
jiduckiess@192 SeminarTesting % cd /Users/jiduckiess/Documents/SeminarTesting/dbunit-demo
mvn test
jiduckiess/.m2/repository/org/xerial/sqlite-jdbc/3.45.3.0/sqlite-jdbc-3.45.3.0.jar)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

Total orders: 1
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.387 s -- in com.eshop.dbunit.EshopDbUnit
Test
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.224 s
[INFO] Finished at: 2026-07-04T00:17:01+07:00
```

Database Rider Evidence

Database Rider evidence 1

```
jiduckiess@192 backend % cd /Users/jiduckiess/Documents/SeminarTesting/data
base-rider-demo
mvn test
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for call
ers in this module
WARNING: Restricted methods will be blocked in a future release unless nati
ve access is enabled

[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.60
6 s -- in com.eshop.databaserider.EshopDatabaseRiderTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.251 s
[INFO] Finished at: 2026-07-04T18:24:40+07:00
[INFO] -----

jiduckiess@192 database-rider-demo %
```

Database Rider evidence 2

```
jiduckiess@192 database-rider-demo % cd /Users/jiduckiess/Documents/Seminar
Testing
sqlite3 backend/database.sqlite "SELECT id, name, email, role FROM users;"
1|Admin User|admin@eshop.com|admin
2|Test User|test@eshop.com|user
jiduckiess@192 SeminarTesting %
```

Ln 280, Col 75

Tonic.ai Evidence

Tonic.ai evidence 1

```
jiduckiess@192 SeminarTesting % sqlite3 -header -csv backend/database.sqlite "SELECT * FROM users;" > tonic-export/source/users.csv
sqlite3 -header -csv backend/database.sqlite "SELECT * FROM products;" > tonic-export/source/products.csv
sqlite3 -header -csv backend/database.sqlite "SELECT * FROM coupons;" > tonic-export/source/coupons.csv
sqlite3 -header -csv backend/database.sqlite "SELECT * FROM orders;" > tonic-export/source/orders.csv
jiduckiess@192 SeminarTesting % ls -la tonic-export/source
total 24
drwxr-xr-x@ 6 jiduckiess  staff  192 Jul  4 12:49 .
drwxr-xr-x@ 3 jiduckiess  staff   96 Jul  4 12:49 ..
-rw-r--r--@ 1 jiduckiess  staff  256 Jul  4 12:49 coupons.csv
-rw-r--r--@ 1 jiduckiess  staff    0 Jul  4 12:49 orders.csv
-rw-r--r--@ 1 jiduckiess  staff  700 Jul  4 12:49 products.csv
-rw-r--r--@ 1 jiduckiess  staff  194 Jul  4 12:49 users.csv
id,name,email,password,role,login_attempts,locked_until,reset_token,shipping_address,phone
1,"Admin User",admin@eshop.com,Admin123!,admin,0,,,,,
2,"Test User",test@eshop.com,Test1234!,user,0,,,,,
```

Tonic.ai evidence 2

```
tonic-export > source > users.csv
1 id,name,email,password,role,login_attempts,locked_until,reset_token,shipping_address,phone
2 1,"Admin User",admin@eshop.com,Admin123!,admin,0,,,,,
3 2,"Test User",test@eshop.com,Test1234!,user,0,,,,,
4
```

Tonic.ai evidence 3

The screenshot shows the Tonic.ai web interface. At the top, there's a navigation bar with options like 'Search workspaces', 'Workspaces', 'Add Data Source', 'Getting Started', 'Book A Demo', and '13 days left in free trial'. Below this, the workspace name 'Duc' is displayed, owned by 'Đức Nguyễn Chí'. A sidebar on the left contains a 'Hello!' message from the AI assistant and a list of actions: 'Review my sensitivity scan results', 'Configure generators to mask sensitive columns', and 'What can Tonic Structural do?'. The main area shows a message 'Could not connect to source database' with a sub-message 'To connect a data source, edit the workspace.' and buttons for 'Edit Workspace Settings' and 'View Docs'. A red-bordered box contains a 'Connection Error' message: 'No Source DB has been configured for this Workspace'. At the bottom left, there's a text input field labeled 'Ask the Tonic Agent...'.

Tonic.ai evidence 4

Model

NAME

name » Name
Type : First Last

EMAIL

email » Email

PASSWORD

password » Constant
Value : MaskedPassword123!

SHIPPING_ADDRESS

shipping_address » Address
Type : Zip Code

PHONE

phone » Phone

Table: users

De-Identify

Preview

id	name	email	password	role	login_attempts	locked_until
1	Leif Kryslinski	shyq@noted.its	MaskedPassword123!	admin	0	
2	Ruthanne Gsell	bcma@qisfo.gnx	MaskedPassword123!	user	0	

Tonic.ai evidence 5

Confirm Generation

Settings

Destination

Generation removes previous versions of the generated files, and creates new versions of the files based on the current generator settings.

☐ Enable Diagnostic Logging

☐ Send Logs to Tonic.ai

☐ Collect Performance Metrics

Run Generation

Want faster generations?

Use the suggested tips to make workspace and database changes that can improve generation performance.

Generation Tips

Tonic.ai evidence 6

Agent

File Groups

Privacy Hub

Database View

Table View

Foreign Keys

Jobs

Schema Changes

Post-Job Actions

Workspace Settings

Jobs

The jobs are grouped by category. To view the jobs in a category, click the category tab.

Learn More

Scheduled Jobs

Schedule your data generation jobs to run at the times that work best for you.

Create Schedule

All Jobs

Data Generation

Sensitivity Scan

Schema

Filter jobs by ID...

Job ID	Type	Status	Submitted	Completed
088f3138-35fc-a97e-abee... Data Pipeline Generation	Schema Retrieval	Completed a few seconds ago	07/04/2026 5:45 PM	07/04/2026 5:45 PM
e611fb86-32e3-9a33-8a3... Data Pipeline Generation	Data Pipeline Generation	Canceled a minute ago	07/04/2026 5:44 PM	07/04/2026 5:45 PM
962c9422-6f22-36b6-003... Data Pipeline Generation	Data Pipeline Generation	Completed a few seconds ago	07/04/2026 5:43 PM	07/04/2026 5:45 PM
e066ee58-1b35-846d-00c... Schema Retrieval	Schema Retrieval	Completed 18 minutes ago	07/04/2026 5:28 PM	07/04/2026 5:28 PM
71ca4e98-8cec-6bde-158... Sensitivity Scan	Sensitivity Scan	Completed 16 minutes ago	07/04/2026 5:28 PM	07/04/2026 5:29 PM
1bf1d0a4-ad3d-775d-9d94... Schema Retrieval	Schema Retrieval	Completed 18 minutes ago	07/04/2026 5:27 PM	07/04/2026 5:27 PM
5a8339f9-ac64-c764-167... Sensitivity Scan	Sensitivity Scan	Completed 17 minutes ago	07/04/2026 5:27 PM	07/04/2026 5:29 PM
2f97e335-0de5-cd11-525e... Schema Retrieval	Schema Retrieval	Completed 24 minutes ago	07/04/2026 5:22 PM	07/04/2026 5:22 PM
5162093f-c0a8-b751-4a66... Sensitivity Scan	Sensitivity Scan	Completed 23 minutes ago	07/04/2026 5:22 PM	07/04/2026 5:22 PM
514a4f8e-10d2-a664-e1d7... Schema Retrieval	Schema Retrieval	Completed 25 minutes ago	07/04/2026 5:20 PM	07/04/2026 5:20 PM

View 10 jobs per page

Showing 1 to 10 of 12

Tonic.ai evidence 7

[Back to All Jobs](#)

Data Pipeline Generation Job Details

Job ID: 962c9422-6f22-36b6-003d-5e8644eb64f3

Copy Job ID

Reports and Logs

Workspace

Untitled Workspace

Owner: ncdudc23@clc.fitus.edu.vn

DB Type: Files

Subsetting: Not In Use

Post-Job Scripts: Not In Use

Webhooks: Not In Use

Worker Version: 1795

Data available for file groups

De-identified files from this data generation job are available for download. Select a file group to download the files.

Download Results

Filter...

coupons

products

orders

users

Start Time

5:45:29 PM

Duration

0m 9s

Job Completed

Completed in a few seconds

Executing test suites

Completed in a few seconds

Showing slowest 2 tasks

Executing Source test suites completed in < 1 second

Executing Destination test suites completed in < 1 second

Total Time: < 1 second

Running pre-job checks

Completed in a few seconds

Applying generators to chained Fks

Completed in a few seconds

Copying passthrough table

Completed in a few seconds

Showing slowest 2 tasks

Copying File Group "coupons" completed in < 1 second

Tonic.ai evidence 8

```
2, test user, test@eshop.com, test123!, user, 0, , , ,
● jiduckiess@192 SeminarTesting % ls -la tonic-export/generated
head -n 5 tonic-export/generated/users.csv
total 32
drwxr-xr-x@ 6 jiduckiess  staff  192 Jul  4 17:50 .
drwxr-xr-x@ 4 jiduckiess  staff  128 Jul  4 17:50 ..
-rw-r--r--@ 1 jiduckiess  staff  256 Jul  4 17:50 coupons.csv
-rw-r--r--@ 1 jiduckiess  staff  106 Jul  4 17:50 orders.csv
-rw-r--r--@ 1 jiduckiess  staff  688 Jul  4 17:50 products.csv
-rw-r--r--@ 1 jiduckiess  staff  228 Jul  4 17:50 users.csv
id,name,email,password,role,login_attempts,locked_until,reset_token,shipping_address,phone
1,Rebbeca  Cobler,mjwp@objcs.ndn,MaskedPassword123!,admin,0,,,97708,
2,Leigha  Batcher,xytg@zxfgo.goy,MaskedPassword123!,user,0,,,99547,
○ jiduckiess@192 SeminarTesting %
```

Tonic.ai evidence 9

```
tonic-export > generated > users.csv
1  id,name,email,password,role,login_attempts,locked_until,reset_token,shipping_address,phone
2  1,Rebbeca  Cobler,mjwp@objcs.ndn,MaskedPassword123!,admin,0,,,97708,
3  2,Leigha  Batcher,xytg@zxfgo.goy,MaskedPassword123!,user,0,,,99547,  Pin selection to
```

Commands used for evidence:

```
cd dbunit-demo
mvn test
```

```
cd database-rider-demo
mvn test
```

```
cd eshop-sut/backend
node database.js
```

12. References

- DbUnit: <https://www.dbunit.org/>
- Database Rider: <https://database-rider.github.io/database-rider/>
- Tonic.ai: <https://www.tonic.ai/>